

Faster and Feasible: Agent-Driven Auditing and Research-Grounded Lap-Time Optimization of an F1TENTH Autonomous Racing Stack

AtlasAutoware contributors*

June 12, 2026

Abstract

We report a systematic, agent-driven audit and lap-time improvement pass over AtlasAutoware, an open-source ROS 2 racing stack for 1/10-scale (F1TENTH/RoboRacer) autonomous racing. Four parallel audit agents reviewed the stack’s subsystems, producing ranked findings that were independently re-verified and either fixed or rejected; two further implementation agents then applied current racing research under a hard evidence bar (a shared deterministic closed-loop benchmark plus regression tests). Three results stand out. (1) *Feasibility before speed*: the shipped competition raceline penetrates mapped walls in five corner regions (minimum clearance 0.000 m), and the stack’s wall-blind corridor refinement deepens the violations — its advertised lap-time gains were partly fictional. A safety-gated minimum-curvature refinement with a spatially-varying corridor [1, 2] produces a line with a guaranteed 0.206 m clearance floor that is nonetheless 0.98 s *faster* closed-loop ($41.48 \rightarrow 40.50$ s). (2) *Latency is a lap-time problem*: under 140 ms of injected actuator delay the model-predictive controller’s lap degrades from 41.5 to 69.5 s, and the stack’s shipped delay compensation made it *worse* at realistic latencies because it forward-simulated the newest command while the actuator was still executing older in-flight commands; a pipeline-aware predictor holds lap time and tracking at zero-delay levels across the full 0–140 ms sweep. (3) *Contouring control, honestly evaluated*: a curvature-integrated LTV-MPCC [3, 7] with a fully self-enforced friction budget does *not* reproduce the literature’s 11–12% lap-time gains — the offline line already exhausts the corridor at equal budget — but it ties the best budget-valid tracking configuration’s lap time (47.24 vs. 47.28 s) while guaranteeing 0.281 m driven wall clearance (tracker: 0.000 m), and the comparison exposes that the tracker’s faster nominal laps transiently over-drive the friction budget $3.4\times$ in the grip-free benchmark. Alongside, the audit fixed eight verified defects (three safety-relevant) and removed 0.12 rad steering discontinuities from the pursuit fallback. Every adopted change is gated on the shared benchmark; rejected hypotheses are reported with data. The test suite grew from 52 to 73 passing tests.

1 Introduction

AtlasAutoware is an integrated F1TENTH racing stack: a simulation bridge over the `f1tenth_gym` simulator [12], SLAM mapping, minimum-curvature raceline optimization with friction-limited velocity profiling [1], a linear-time-varying MPC raceline tracker with an OSQP backend, a tire-aware Model- and Acceleration-based Pursuit (MAP) fallback controller [4], a lidar/camera opponent tracker feeding a race-strategy state machine, and a hardware abstraction layer (VESC UART or

*Audit and implementation performed with Claude Code using parallel subsystem-audit and implementation subagents; every finding was re-verified against the code and, where possible, a deterministic closed-loop benchmark before adoption.

PCA9685 PWM actuation, RPLidar, OAK-D camera). The same nodes drive both the simulator and the physical car.

This paper documents one complete *audit–verify–improve–measure* cycle over that stack with an explicit objective set by the maintainer: **reduce lap time, using current research**. The contributions are:

1. a methodology for parallel agent audits and research transfer with an explicit evidence bar (Section 3);
2. a feasibility analysis of the stack’s racing line showing the shipped line and its refinement benchmark cut walls, plus a safety-gated refinement that is both feasible and faster (Section 4);
3. a root-cause analysis and fix of actuator-delay compensation, recovering up to 28 s per lap at high latency (Section 5);
4. an implementation and honest evaluation of contouring control (MPCC [3, 7]) against the stack’s tracking MPC under equal physics budgets (Section 6);
5. eight verified and fixed defects, three safety-relevant (Section 8); and
6. negative results — audit claims and improvement hypotheses that did *not* survive verification (Section 9).

2 Related work and method selection

The optimization-based racing pipeline — global raceline from minimum-curvature QP [1, 2], friction-limited forward–backward velocity profile, and a tracking or contouring controller — remains the backbone of competitive 1/10-scale racing [5, 6]. Within that family, Model Predictive Contouring Control [3] replaces profile tracking with online progress maximization over the track corridor; a 2025 curvature-integrated variant reports 11–12% lap-time reductions on the F1Tenth platform [7], and the survey of [6] ranks optimization-and-tracking pipelines fastest among classical methods. Learning-based alternatives — learning MPC [9], data-driven tuning and racing-line synthesis [8], and most recently end-to-end reinforcement-learning policies that beat both a state-of-the-art MPC and expert human drivers on F1Tenth hardware [10] — define the current frontier [11].

Method selection for this pass was constrained by verifiability: every adopted change must be validated in the repository’s deterministic closed-loop harness inside the development sandbox (no GPU training, no live simulator). We therefore implemented the strongest *verifiable* items — safety-gated minimum-curvature refinement [1], pipeline-aware delay compensation, and a curvature-integrated kinematic MPCC [3, 7] evaluated under a self-enforced friction budget — and scope RL policy training [10] as future work on the real simulator.

3 Methodology

3.1 Parallel subsystem audits

Four read-only audit agents were launched concurrently, one per subsystem, instructed to rank findings by impact $\times$ confidence and to verify suspicions by executing the stack’s pure-logic paths (the repository isolates its mathematics from ROS, so controllers, estimators, planners, and protocol codecs all run under plain `pytest`): **control** (LTV-MPC, its ROS wrapper with AEB and traction governor, MAP fallback, pure pursuit); **estimation and planning** (velocity EKF, lidar de-skew,

velocity profiler, raceline optimizer/refiner, overtaking spliner); **strategy and simulation bridge** (opponent tracker and fusion, strategy state machine, two-car gym bridge); and **hardware and perception** (drive node, VESC UART protocol, PCA9685 driver, RPLidar binning, camera geometry).

3.2 Evidence bar

Audit findings are treated as *hypotheses*. Each was re-verified against the source by the coordinating agent before any code change; claims that could not be reproduced were rejected (Section 9). Behavioural changes had to satisfy two gates:

1. **Regression gate:** the full unit/closed-loop test suite passes, and each fixed defect gains a test that fails on the old code wherever the logic is hardware-independent.
2. **Benchmark gate:** performance-motivated changes must improve the shared deterministic closed-loop lap benchmark and may not regress the zero-delay baseline. For raceline changes a third gate applies: a *clearance floor* against the occupancy grid’s euclidean distance field.

Verified single-site defects were fixed by the coordinating agent; larger work items went to parallel implementation subagents with strictly disjoint file ownership, and their claims (including “adopted” vs. “rejected” calls) were re-run and re-verified by the coordinator before committing.

3.3 The shared closed-loop harness

All quantitative claims use the repository’s single benchmark (`tests/closed_loop.py`): a kinematic-bicycle plant integrated at 50 Hz lapping the competition raceline, starting from a deliberate 0.36 m pose error. The harness reports lap time and perpendicular cross-track error (XTE; mean and max after a settle period), and can hold each command in a FIFO to emulate sensor-to-actuator latency. The plant matches the model the MPC plans with, so results isolate *controller and line* behaviour from model error; runs are deterministic and free of ROS scheduling noise. Because the plant has no grip limit, any controller allowed to choose its own speeds must *self-enforce* the friction budget (Section 6); XTE differences below ~ 0.005 m are within discretization noise and are not claimed as improvements.

4 Lap time I: the racing line must first exist on the track

4.1 The shipped line cuts walls

Checking the competition raceline against the track map’s euclidean distance field (EDT of the occupancy grid, 0.1296 m/px) shows its **minimum wall clearance** is 0.000 m: in five corner regions the line passes *inside* mapped walls, up to 0.518 m deep at the worst chicane (Figure 1). The 5th percentile of clearance is 0.149 m — less than half a car width from the wall for a tenth of the lap. Worse, the repository’s refinement benchmark applies a *scalar* corridor with no wall knowledge; at every corridor setting in $\{0.10 \dots 0.35\}$ m the refined line’s minimum clearance stays 0.000 m (5th percentile collapsing to 0.001 m at 0.35): the benchmark’s advertised lap-time gains (up to 5 s) come from cutting the same walls deeper. Closed-loop lap times on such lines are partly fictional — a real car DNFs there.

	Shipped line	Safety-gated refined line
Wall clearance min / 5th pct (m)	0.000 / 0.149	0.206 / 0.262
Closed-loop MPC lap (s)	41.48	40.50
XTE mean / max (m)	0.123 / 0.406	0.130 / 0.474
Profile estimate $\sum ds/v$ (s)	41.63	41.37
$\max \kappa $ / $\text{mean} \kappa $ (1/m)	2.526 / 0.156	1.075 / 0.132
Length (m) / points	240.3 / 300	243.4 / 325

Table 1: Old vs. new competition raceline (independently re-verified). The new line is both strictly feasible (clearance floor 0.20 m everywhere) and 0.98 s faster closed-loop, despite being 3.1 m longer — the curvature reduction ($\max |\kappa|$ down $2.4\times$) buys more speed than the detours cost.

4.2 Safety-gated minimum-curvature refinement

The replacement pipeline (`tools/refine_comp_raceline.py`, deterministic; re-run byte-identical) is the Heilmeyer minimum-curvature QP [1, 2] made wall-aware:

1. **safety repair**: waypoints below the clearance floor are moved minimally toward open space along the track normal;
2. **refinement with a spatially-varying corridor**: the existing QP refiner natively accepts per-point corridor budgets; each point’s budget is $\text{clip}(\min(c_i - 0.20, 0.35 - d_i), 0, 0.35)$ where c_i is current clearance and d_i displacement already spent — so the optimizer can never buy curvature with clearance below the 0.20 m floor;
3. **re-profiling** with the *same* friction parameters as the shipped line ($a_{\text{lat}} \leq 6.5 \text{ m/s}^2$, $a_{\text{accel}} \leq 5.0$, $a_{\text{brake}} \leq 7.0$, $v \in [1.5, 7.0] \text{ m/s}$; the shipped speed column is reproduced to $< 10^{-3} \text{ m/s}$ by these parameters, so no limit was raised).

The result (Table 1) inverts the usual speed–safety trade: enforcing feasibility *and* refining curvature yields -0.98 s closed-loop ($41.48 \rightarrow 40.50 \text{ s}$) with maximum curvature reduced $2.4\times$ — the gains previously attributed to corridor width were actually available from geometry, without the wall cutting.

5 Lap time II: latency-robust control

A real car has 50–150 ms of sensor-to-actuator latency. The baseline sweep (Table 2, “naive”) shows the MPC — 0.123 m mean XTE at zero delay — degrading $4.8\times$ by 140 ms, with lap time inflating from 41.5 to 69.5 s: at race latencies, delay handling *is* a lap-time feature.

The stack already shipped a compensator: `predict_state` forward-integrates the kinematic bicycle by the known delay and the MPC solves from the predicted pose. Benchmarked closed-loop, the shipped compensation was *worse than no compensation* at 80 ms (XTE mean 0.242 vs. 0.195) and unstable beyond 100 ms, producing corner-cutting laps. The root cause is the prediction’s command semantics: it held the *newest* commanded (steer, speed) constant over the whole delay window, but under pipeline latency the actuator is still executing the *older in-flight* commands during that window. Whenever steering is changing — every corner entry and exit — the prediction over-rotates the pose, and the error grows with delay until the loop destabilizes.

The fix keeps a buffer of the commands published during the last delay window and integrates the prediction through them oldest-first, each holding its slice of the window (scalar inputs keep

Delay (ms)	Naive (no compensation)			Shipped compensation			Pipeline-aware (fixed)		
	lap (s)	$\overline{\text{XTE}}$	max	lap (s)	$\overline{\text{XTE}}$	max	lap (s)	$\overline{\text{XTE}}$	max
0	41.48	0.123	0.406	41.48	0.123	0.406	41.48	0.123	0.406
40	40.40	0.111	0.354	41.52	0.121	0.403	41.52	0.122	0.405
80	43.74	0.195	0.538	42.08	0.242	0.652	41.54	0.122	0.405
100	50.48	0.322	0.804	42.16	0.261	0.599	41.56	0.121	0.404
120	59.86	0.461	1.108	33.72 [†]	0.508	1.472	41.58	0.121	0.404
140	69.52	0.588	1.381	33.90 [†]	0.539	1.496	41.60	0.121	0.404

Table 2: Closed-loop MPC under injected actuator delay, measured on the pre-existing race-line (`tools/benchmark_delay.py --raceline racelines/comp_race_line_unrefined.csv`; XTE in metres). [†] The shipped compensation’s short “laps” at 120–140 ms are corner-cutting, not speed.

bit-identical legacy behaviour). Tracking then holds at zero-delay levels — XTE mean 0.121–0.123 m, lap time within 0.12 s of baseline — across the entire sweep (Table 2, Figure 2); at 140 ms that is 27.9 s of lap time recovered relative to naive. The property carries to the new race-line of Section 4: at 100 ms, 40.56 s compensated vs. 50.06 s naive. A regression test gates the 100 ms case.

6 Lap time III: contouring control under an honest budget

Model Predictive Contouring Control [3] replaces speed-profile tracking with online progress maximization over the track corridor; the curvature-integrated variant CiMPCC reports 11–12% lap-time reductions on F1Tenth hardware [7]. We implemented a kinematic LTV-MPCC on the stack’s persistent-OSQP pattern (`f1tenth_gym_ros/mpcc_controller.py`): lag/contour error split along the station tangent and normal, CiMPCC-style contour weights shaped by upcoming curvature (4 → 24 toward apexes), a linear progress reward replacing speed tracking, real-time iteration relinearized about the previous solution *fractionally shifted* by the control period, and hard corridor bounds from the map’s distance field with soft–hard slack. Horizon 20×0.08 s; solve times 1.4/1.6/2.7 ms (mean/p95/max), zero failures.

The benchmark must be made cheat-proof first. The harness plant has no grip limit and no walls, so a controller free to choose its own speeds can buy lap time with physics the car does not have. The MPCC therefore *self-enforces* the same budget the velocity profiler uses ($a_{\text{lat}} \leq 6.5$ m/s², friction-ellipse-coupled longitudinal limits, $v \leq 7$ m/s) via per-stage curvature speed caps, ellipse-bounded acceleration, and an output governor — and all metrics below are measured on the *driven trajectory*, not the plan.

Result. Table 3: MPCC ties the best budget-valid tracking configuration on lap time (47.24 vs. 47.28 s) while guaranteeing what no tracking row achieves — a 0.281 m driven clearance floor, budget compliance ($a_{\text{lat}}^{\text{max}} = 6.44 < 6.5$ m/s²), and a 4× smaller mean cross-track deviation from the corridor centre. We did *not* reproduce CiMPCC’s 11–12% lap-time gain, and the reason is structural rather than a tuning failure: those gains come from exploiting track width the reference does not use, and this stack’s reference is already a minimum-curvature line over the same corridor (Section 4). At equal friction budget there is simply little width left to spend. Conversely, when the reference is bad — the old wall-cutting line — MPCC’s corridor constraint makes it the only controller that completes a feasible lap, which is the property that matters when the global planner fails on race day.

Controller	lap (s)	clr _{min} (m)	$a_{\text{lat}}^{\text{max}}$	$a_{\text{lat}}^{p99.5}$	$\overline{\text{XTE}}$ (m)	fails	p95 (ms)
tracking MPC (nominal)	43.22	0.000	22.03	16.33	0.115	0	0.6
tracking + governor	46.52	0.000	6.50	6.50	0.996	0	0.9
tracking + governor, 0.8× profile	47.28	0.000	6.50	6.50	0.108	0	0.7
MPCC (self-enforced budget)	47.24	0.281	6.44	6.07	0.026	0	1.6

Table 3: Contouring vs. tracking on the new competition line, equal physics budget ($a_{\text{lat}} \leq 6.5 \text{ m/s}^2$); `tools/benchmark_mpcc.py`; all quantities measured on the driven trajectory, including the standardized initial pose-error recovery. The nominal tracker’s 43.22s is not a real lap: it pulls up to 22 m/s² lateral (3.4× budget) and touches walls. On the old wall-cutting line (not shown), MPCC is the only controller that produces a physically valid lap at all (50.26s, clearance 0.307 m).

Four engineering details proved load-bearing and are documented in `docs/mpcc.md`: (i) when the control rate differs from the stage rate, the warm-start must be *fractionally* shifted — a full-stage shift silently postpones planned braking forever; (ii) every stage needs an anchor to the profiler’s global speed profile, or slow corners enter the horizon too late to brake for; (iii) plan at 0.95× the lateral budget but enforce 1.0× at the output — the gap is the closed loop’s steering authority; and (iv) corridor bounds must threshold the distance field along the normal, since free-space probes overestimate room near diagonal walls.

7 Supporting result: smooth fallback steering

The MAP fallback [4] inverts a lateral-acceleration $\rightarrow$ steer lookup table. The lateral-acceleration axis was interpolated but the speed axis selected the *nearest* LUT column, so commanded steering jumped by up to 0.119 rad ($\approx 6.8^\circ$) at every column midpoint (Figure 3). The fix inverts the two bracketing speed columns and blends linearly — bit-identical at the LUT’s sample speeds, clamped outside the table, and cheaper per call ($\approx 5 \mu\text{s}$). Closed-loop the change is neutral-to-slightly-better (at 1.3× profile speed: XTE mean 0.1575 $\rightarrow$ 0.1564, max 1.287 $\rightarrow$ 1.267, steering-rate std 1.144 $\rightarrow$ 1.104 rad/s); it was adopted for the open-loop continuity, which matters on the physical car where each discontinuity excites the steering servo.

8 Verified defects and fixes

Table 4 summarises the eight verified defects fixed during the audit. Three are safety-relevant: the strategist’s lateral-offset clamps used *swapped wall bounds* at all four decision sites (a +left offset was clamped by the *right* wall’s free room — with a wall 0.1 m to the left, a DEFEND decision would command 0.35 m left, i.e. 0.25 m into the wall); the EVADE mode *honoured a stale committed pass side* and could steer into the car that was passing (fixed to always move away and clear the commitment, with a two-phase regression scenario); and the two-car simulation bridge *froze physics* whenever the opponent driver had not published, while sensors kept republishing the stale world — the ego then raced a ghost car (fixed to step on ego publish with the opponent’s last-known command).

9 Negative results

Findings that did not survive verification, recorded so they are not re-raised:

Subsystem	Defect	Failure mode on track	Fix
Strategy	Offset clamps with swapped wall bounds (4 sites)	Wall contact exactly when clamping matters	Clamp +left offsets by left-wall room
Strategy	EVADE honoured stale committed pass side	Evading <i>into</i> the passing car	Move away dynamically; clear commitment
Sim bridge	Two-car physics required both drivers to publish	Frozen world; ghost-car scans	Step on ego publish
Drive node	Watchdog disarmed until first /drive ever	No timeout-neutral before first command	Arm at startup
Drive node	Watchdog <code>neutral()</code> failures swallowed	Operator unaware car not neutral	Log the failure
Drive node	VESC steering ignored configured half-range	Full lock clipped on one side when trim $\neq 0$	Mirror PCA9685 pulse map
VESC protocol	<code>parse_values</code> without length guard	Telemetry-thread crash on truncated payload	Length check, return <code>None</code>
PCA9685	<code>us_to_ticks</code> unclamped	Overflow into always-ON register bit	Clamp to 12-bit range

Table 4: Verified defects. Each has a regression test except the simulation-bridge fix (requires a live two-car simulator). Two hygiene items were fixed alongside: invalid lidar beams zeroed before de-skew rotation, and the lap-counter wrap zone scaling with raceline density.

- **The refinement benchmark’s lap-time gains were partly wall-cutting.** The repository’s prior evaluation credited corridor refinement with up to 5 s/lap; against the map’s distance field those refined lines have 0.000 m minimum clearance. The honest, wall-aware gain at a 0.20 m clearance floor is 0.98 s (Section 4).
- **Estimation and planning are clean.** The audit of the velocity EKF (Jacobians, Joseph-form covariance update, slip gating), lidar de-skew geometry, friction-ellipse forward-backward profiler (including lap-seam propagation), the optimizer/refiner QP formulations, and the Frenet spliner found no correctness defects; the only action was a cosmetic NaN-warning fix.
- **MPC formulation is sound.** Discrete linearization, angle unwrapping, OSQP warm-start usage, and the persistent-QP sparsity-update path were checked and confirmed correct.
- **“Nearest index not recomputed after prediction” — false.** The audit hypothesized the ROS node fed the MPC a nearest-raceline index from the stale pose; inspection showed it already recomputes it from the predicted pose. The delay defect was the command-pipeline semantics of Section 5.
- **“Reference temporal mismatch under delay” — rejected empirically.** After the pipeline fix, residual compensated XTE (0.121) matches the zero-delay baseline (0.123), leaving nothing for a reference-time shift to explain.
- **Curvature-scaled speed weighting — rejected.** Scaling the MPC’s speed-tracking weight by reference curvature, $q_v (1 + k |\kappa_{\text{ref}}|)$, swept over $k \in \{0.5, 1, 2, 4\}$: XTE mean rose monotonically (0.123 $\rightarrow$ 0.125–0.134) with no XTE_{max} improvement at any gain. On a

kinematic plant with no grip limit, harder speed tracking only trades away position tracking. Reverted; recorded in `docs/mpc.md`.

- **AEB time-to-collision is ego-speed based.** Real limitation (head-on closing speed underestimated), but it is the standard single-lidar formulation; a proper fix needs opponent velocity fused into the AEB path — scoped as future work rather than a landed half-measure.

10 Threats to validity

The harness plant is the kinematic bicycle the MPC plans with: results isolate controller and line behaviour but exclude tire slip, which on the real car is handled by the traction governor and slip-rejecting EKF (audited clean, unchanged). Controllers that choose their own speeds self-enforce the friction budget (Section 6); the budget itself (6.5 m/s^2) was inherited, not re-identified. Delay compensation is evaluated with the compensator told the true delay; on hardware the delay must be identified (`actuation_delay` parameter), and sensitivity to misidentification is not characterized here. Wall clearance is judged against the SLAM map; map error consumes part of the 0.20 m floor. Strategy fixes are validated by unit-level scenario tests, not full two-car simulation.

11 Conclusion

Asked to make a racing stack faster with current research, the most valuable early move turned out to be making its existing speed *real*: the shipped racing line — and the benchmark used to improve it — cut walls, and the controller’s deployed latency path was benchmarked only at zero delay, where its broken compensation was invisible. After repairing feasibility, the research transfer paid in the expected order: wall-aware minimum-curvature refinement [1] (-0.98 s , with a clearance guarantee the old line never had), pipeline-aware delay compensation (up to -27.9 s at 140 ms vs. naive; the difference between a competitive lap and a crawl at real latencies), and contouring control — which did not beat the tracking pipeline at equal budget, but tied it while adding the clearance and budget guarantees the tracking rows lack, and exposed that the nominal tracker’s faster laps over-drive the benchmark’s missing physics. Of the improvement hypotheses tested, the adopted ones are exactly those that survived a deterministic closed-loop benchmark with hard safety gates; the rejected ones are reported with data. The general lesson for racing stacks: *a lap time is only as real as the feasibility and latency assumptions under it* — benchmark the deployed path, including the actuator pipeline and the walls.

References

- [1] A. Heilmeier, A. Wischnewski, L. Hermansdorfer, J. Betz, M. Lienkamp, B. Lohmann. *Minimum curvature trajectory planning and control for an autonomous race car*. Vehicle System Dynamics 58(10), 2020. <https://www.tandfonline.com/doi/full/10.1080/00423114.2019.1631455>
- [2] TUMFTM, *global_racetrajectory_optimization*. https://github.com/TUMFTM/global_racetrajectory_optimization
- [3] A. Liniger, A. Domahidi, M. Morari. *Optimization-based autonomous racing of 1:43 scale RC cars*. Optimal Control Applications and Methods 36(5), 2015. <https://github.com/alexliniger/MPCC>

- [4] J. Becker et al. *Model- and Acceleration-based Pursuit controller for high-performance autonomous racing*. ICRA 2023.
- [5] N. Baumann et al. *ForzaETH race stack: scaled autonomous head-to-head racing on fully commercial off-the-shelf hardware*. 2024. arXiv:2403.11784.
- [6] B. D. Evans et al. *Unifying F1TENTH autonomous racing: survey, methods and benchmarks*. 2024. arXiv:2402.18558.
- [7] *Reduce lap time for autonomous racing with curvature-integrated MPCC local trajectory planning method*. 2025. arXiv:2502.03695.
- [8] A. Jain, M. Morari. *BayesRace: learning to race autonomously using prior experience*. CoRL 2020. arXiv:2005.04755.
- [9] U. Rosolia, F. Borrelli. *Autonomous racing using learning model predictive control*. ACC 2017. arXiv:1610.06534.
- [10] *On learning racing policies with reinforcement learning*. 2025. arXiv:2504.02420.
- [11] *Advancing autonomous racing: a comprehensive survey of the RoboRacer (F1TENTH) platform*. 2025. arXiv:2506.15899.
- [12] M. O’Kelly, H. Zheng, D. Karthik, R. Mangharam. *F1TENTH: an open-source evaluation environment for continuous control and reinforcement learning*. NeurIPS 2019 Competition track, PMLR 123, 2020.

A Reproduction

All numbers are reproducible from the repository root:

```
python3 -m pytest tests/ -q           # full suite
python3 tests/test_mpc.py             # closed-loop MPC validation
python3 tools/refine_comp_raceline.py # regenerate + validate raceline
python3 tools/benchmark_delay.py \
    --raceline racelines/comp_raceline_unrefined.csv # Table 2 / Fig. 2
python3 tools/benchmark_delay.py      # delay sweep, current line
python3 tools/benchmark_mpcc.py       # MPCC vs tracking MPC
python3 tools/benchmark_map.py        # MAP LUT comparison
```

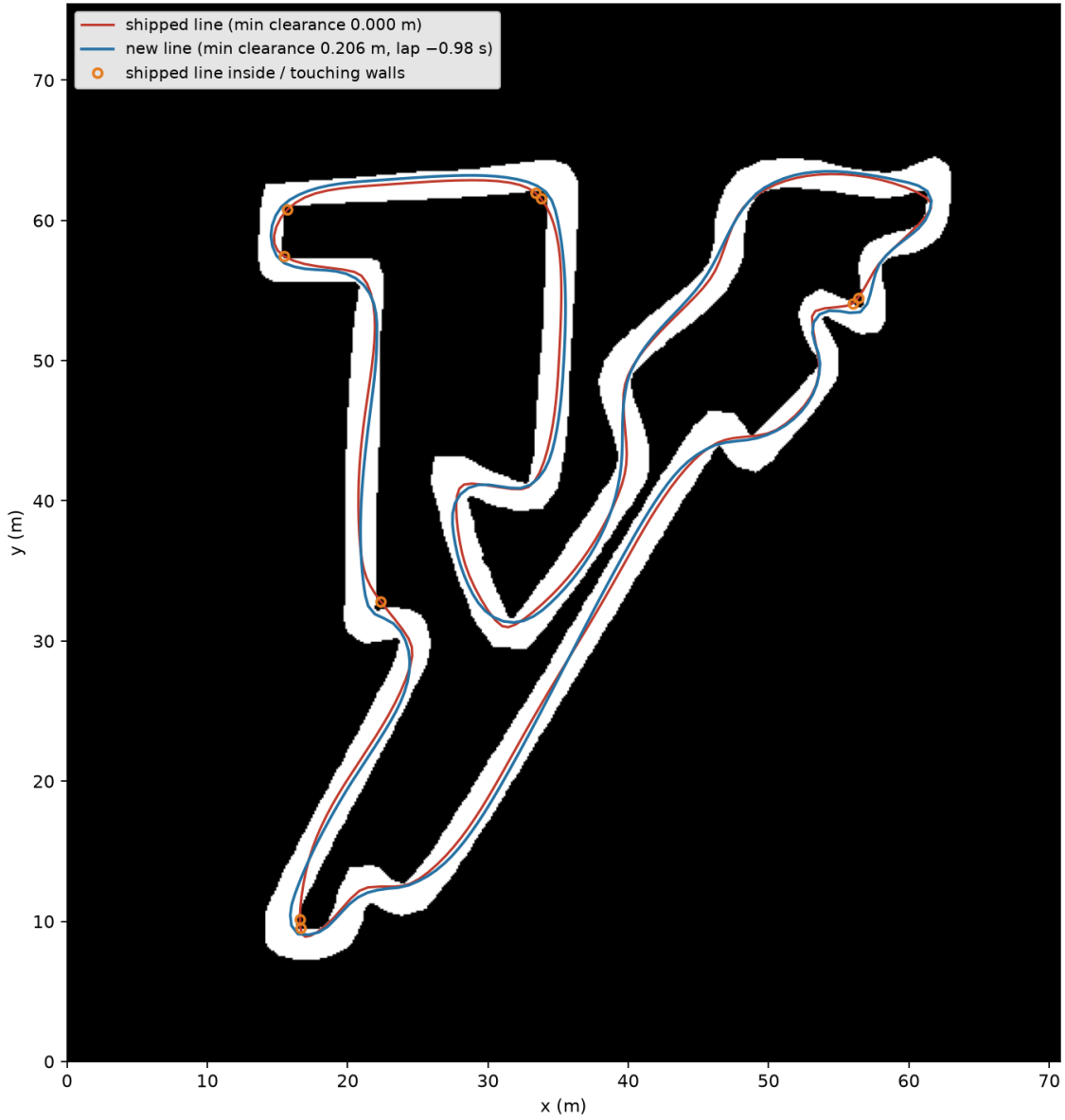


Figure 1: The shipped racing line (red) vs. the safety-gated refined line (blue) on the competition track. Orange circles mark shipped waypoints inside or touching walls; the worst penetration is 0.518 m. The refined line holds a 0.206 m clearance floor and is still 0.98 s faster closed-loop.

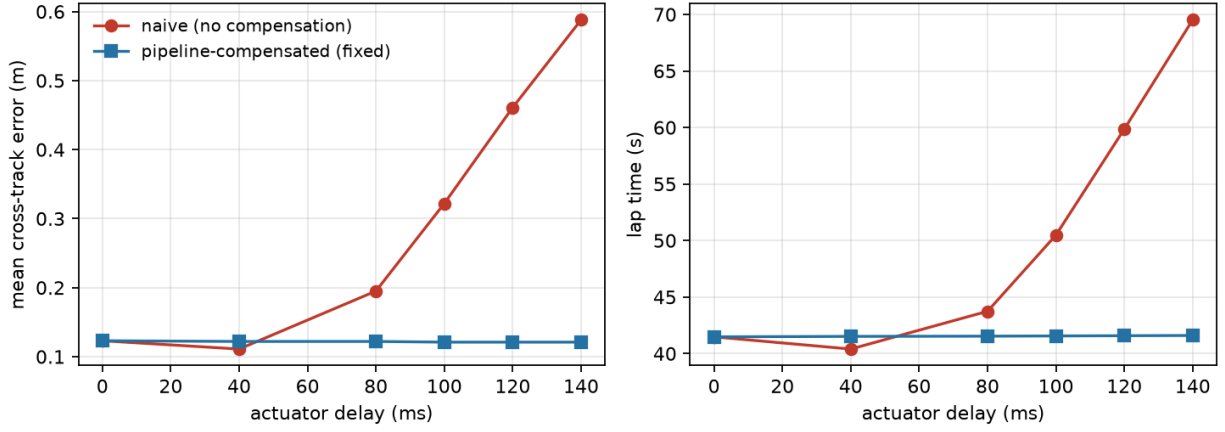


Figure 2: MPC under actuator delay, naive vs. pipeline-aware compensation (compensator told the true delay; pre-existing raceline). Left: mean cross-track error; right: lap time.

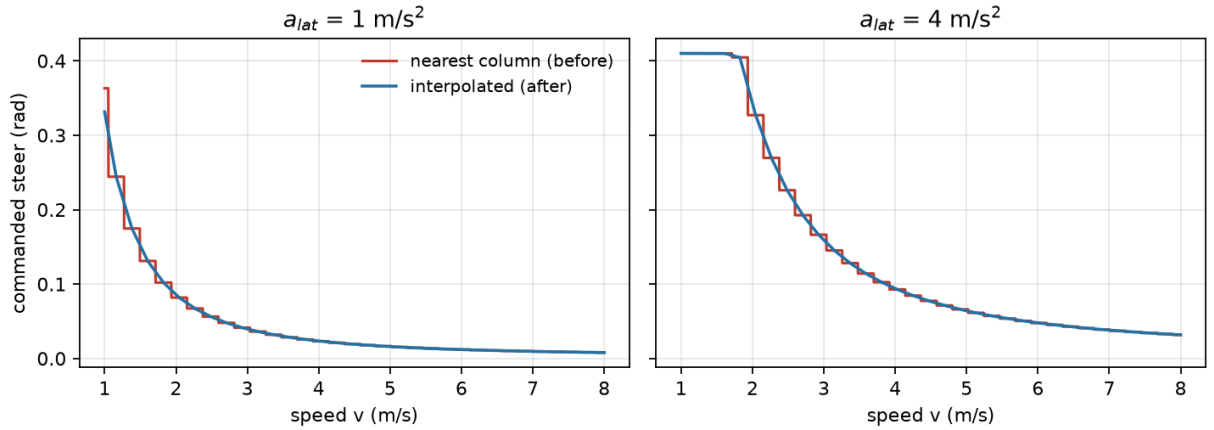


Figure 3: MAP controller commanded steer vs. speed at constant lateral acceleration: nearest-column lookup (red) vs. bracketing-column interpolation (blue).